Revisão de Go: Concorrência I e II

exercicios7 e exercicios8: goroutines, channels, WaitGroup, Mutex, select
Gerado a 4 de novembro de 2026

Como usar este documento

Objetivo: rever e **compreender** a concorrência em Go antes de começar o projeto da CLI de tarefas (sábado e domingo). Cada exemplo é um programa completo, que já foi corrido com `go run -race` e passa sem avisos.

- **Quinta (hoje/amanhã):** Parte A (exercicios7). Lê o modelo mental, copia cada exemplo, corre-o, muda uma coisa e prevê o resultado antes de correr.
- **Sexta:** Parte B (exercicios8) e Parte C (dúvidas). Faz os desafios da Parte D sem copiar.
- **Sábado e domingo:** projeto da CLI (`tarefas-cli/PLANO.md`).

Onde está o código: na pasta `GO BASIC/revisao/` há uma subpasta por exemplo (`e01` ... `e13`). Para correr: `cd revisao && go run -race ./e01`. **Copia do ficheiro `.go`**, não do PDF - o PDF troca alguns símbolos (aspas, hífenes) e o código copiado pode não compilar.

Método que funciona: (1) lê o exemplo; (2) tapa a solução e tenta escrevê-lo de memória; (3) corre com `-race`; (4) altera algo (número de goroutines, remove um `close`, tira o buffer) e vê o que quebra. Entender porque é que quebra é o que fixa a matéria.

Parte A - Concorrência I (exercicios7)

A.1 Modelo mental

- **Goroutine** = função a correr "ao lado" das outras (`go f()`). É barata: podes ter milhares. A função `main` NÃO espera por elas: se `main` acabar, o programa acaba.
- **Channel sem buffer** (`make(chan T)`) = encontro. Quem envia espera por quem recebe, e vice-versa. Sincroniza as duas goroutines.
- **Channel com buffer** (`make(chan T, n)`) = caixa com n lugares. O envio só bloqueia quando a caixa está cheia; a receção só quando está vazia.
- **close(ch)** = "não vou enviar mais". Quem recebe ainda lê o que sobra no buffer; depois recebe o valor zero com `ok == false`. `for v := range ch` termina sozinho quando o channel fecha e esvazia.
- **Regra do close:** só quem ENVIA fecha, e fecha uma só vez. Enviar para um channel fechado dá panic. Fechar um channel não é obrigatório para libertar memória, mas é obrigatório se alguém faz `range` nele.
- **Direções:** `chan<- T` só envia, `<-chan T` só recebe. Servem para o compilador te proteger.
- **sync.WaitGroup** = contador de goroutines em curso. `Add(1)` ANTES de lançar, `defer Done()` dentro, `Wait()` fora (depois do ciclo). Passa-se SEMPRE por ponteiro.
- **Deadlock** = todas as goroutines à espera de algo que nunca vem. Go deteta quando TODAS estão presas ("all goroutines are asleep"); quando só algumas estão presas é um **leak**, sem erro nenhum.

A.2 Quando uso o quê?

- Preciso do resultado de uma goroutine -> channel (um valor) ou slice com índice próprio por goroutine.
- Preciso só de esperar que várias acabem -> WaitGroup.
- Fluxo contínuo de valores entre etapas -> channels + close + range (pipeline).
- Limitar quantas coisas correm ao mesmo tempo -> worker pool (n trabalhadores) ou semáforo (channel com buffer).
- Juntar vários produtores num só consumidor -> fan-in (Merge).

A.3 Exemplo 1 - goroutines, channel com e sem buffer (e01)

Generaliza o exercício 1 (`SomarParalelo`) para N partes e mostra um "ping-pong" com channels sem buffer e um channel só para sinalizar (`struct{}`). Repara em: `chan<-` na função da goroutine; buffer do tamanho certo para ninguém ficar preso; receber **exatamente** tantas somas quanto as goroutines lançadas.

```
package main

import "fmt"

// somaParcial soma um pedaço da slice e envia o total pelo channel.
// "out chan<- int" = só pode ENVIAR (o compilador impede receber).
func somaParcial(pedaco []int, out chan<- int) {
    total := 0
    for _, v := range pedaco {
        total += v
    }
    out <- total
}

// SomaEmPartes generaliza o SomarParalelo (ex. 1) para N partes.
func SomaEmPartes(nums []int, partes int) int {
    if len(nums) == 0 || partes < 1 {
        return 0
    }
    if partes > len(nums) {
        partes = len(nums)
    }
    out := make(chan int, partes)            // buffer: nenhuma goroutine fica presa
    tam := (len(nums) + partes - 1) / partes // divisão arredondada para cima

    lancadas := 0
    for ini := 0; ini < len(nums); ini += tam {
        fim := min(ini+tam, len(nums))
        go somaParcial(nums[ini:fim], out)
        lancadas++
    }

    total := 0
    for i := 0; i < lancadas; i++ { // recebe exatamente tantas quantas lançou
        total += <-out
    }
    return total
}

// pingPong mostra channels SEM buffer: cada envio espera por um receptor.
func pingPong() {
    ping := make(chan string)
    pong := make(chan string)

    go func() {
        for msg := range ping { // termina quando ping for fechado
            pong <- msg + " -> recebido"
        }
        close(pong)
    }()

    for _, m := range []string{"a", "b", "c"} {
        ping <- m
        fmt.Println(<-pong)
    }
    close(ping)
    _, ok := <-pong // pong fechado: devolve valor zero e ok == false
    fmt.Println("pong ainda aberto?", ok)
}

// esperarTerminar usa um channel só para SINALIZAR (struct{} ocupa 0 bytes).
func esperarTerminar() {
    feito := make(chan struct{})
    go func() {
        fmt.Println("a trabalhar...")
        close(feito) // fechar acorda quem estiver à espera
    }()
    <-feito
    fmt.Println("terminou")
}

func main() {
    nums := []int{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
    fmt.Println(SomaEmPartes(nums, 3), SomaEmPartes(nums, 1), SomaEmPartes(nums, 50), SomaEmPartes(nil, 4))
    pingPong()
    esperarTerminar()
}
```

Saída real (`go run -race ./e01`):

```
55 55 55 0
a -> recebido
b -> recebido
c -> recebido
pong ainda aberto? false
a trabalhar...
terminou
```

Para pensar: (1) Porque é que `out` tem buffer `partes`? O que acontece se for sem buffer? (2) Porque é que o `for` final recebe `lancadas` vezes e não `partes`? (3) O que acontece ao programa se esquecer o `close(feito)` em `esperarTerminar`?

A.4 Exemplo 2 - produtor, close e range (e02)

Mostra o contrato do `close`: leitura depois do close devolve o que sobra e depois `0, false`; enviar depois do close dá panic (apanhado aqui com `recover` só para o veres).

```
package main

import "fmt"

// Gerar devolve um channel SÓ DE LEITURA com 1..n e fecha no fim.
func Gerar(n int) <-chan int {
    ch := make(chan int)
    go func() {
        defer close(ch) // garante o close mesmo que o código mude
        for i := 1; i <= n; i++ {
            ch <- i
        }
    }()
    return ch // devolve logo; a goroutine continua a produzir
}

// Consumir lê tudo até o channel fechar.
func Consumir(in <-chan int) (soma int) {
    for v := range in {
        soma += v
    }
    return soma
}

func main() {
    fmt.Println("soma 1..5:", Consumir(Gerar(5)))
    fmt.Println("soma 1..0:", Consumir(Gerar(0))) // fecha logo, 0 valores

    // Channel com buffer: os valores ficam guardados depois do close.
    ch := make(chan int, 2)
    ch <- 10
    ch <- 20
    close(ch)

    v, ok := <-ch
    fmt.Println(v, ok) // 10 true
    v, ok = <-ch
    fmt.Println(v, ok) // 20 true
    v, ok = <-ch
    fmt.Println(v, ok) // 0 false  (fechado e vazio)

    // Enviar para channel fechado = panic. Só QUEM ENVIA deve fechar.
    func() {
        defer func() { fmt.Println("recuperado:", recover()) }()
        ch <- 30
    }()
}
```

Saída real (`go run -race ./e02`):

```
soma 1..5: 15
soma 1..0: 0
10 true
20 true
0 false
recuperado: send on closed channel
```

Para pensar: o `defer close(ch)` dentro da goroutine garante o close mesmo que adiciones um `return` antecipado mais tarde. Porque é que o `Gerar` devolve `<-chan int` e não `chan int`?

A.5 Exemplo 3 - WaitGroup e resultados por posição (e03)

Duas formas de recolher resultados SEM Mutex: cada goroutine escreve só na sua posição `i` de uma slice criada antes. É seguro porque posições diferentes são variáveis diferentes. O `DividirTodos` guarda também o erro de cada posição. O último exemplo mostra o `WaitGroup` passado por ponteiro.

```
package main

import (
    "fmt"
    "sync"
)

// QuadradosParalelo: cada goroutine escreve só na SUA posição i,
// por isso não há corrida de dados (índices diferentes).
func QuadradosParalelo(nums []int) []int {
    res := make([]int, len(nums))
    var wg sync.WaitGroup

    for i, v := range nums {
        wg.Add(1) // ANTES de lançar a goroutine, nunca dentro dela
        go func() {
            defer wg.Done()
            res[i] = v * v // Go >= 1.22: i e v são novos em cada volta
        }()
    }
    wg.Wait()
    return res
}

// Resultado guarda valor E erro por posição: assim não se perde a ordem.
type Resultado struct {
    Valor int
    Erro  error
}

// DividirTodos faz 100/d para cada d. Divisão por zero vira erro, não panic.
func DividirTodos(divisores []int) []Resultado {
    res := make([]Resultado, len(divisores))
    var wg sync.WaitGroup

    for i, d := range divisores {
        wg.Add(1)
        go func() {
            defer wg.Done()
            if d == 0 {
                res[i] = Resultado{Erro: fmt.Errorf("divisor %d: divisão por zero", i)}
                return
            }
            res[i] = Resultado{Valor: 100 / d}
        }()
    }
    wg.Wait()
    return res
}

// trabalhador recebe o WaitGroup POR PONTEIRO. Passar por valor copia-o
// e o Wait ficaria à espera para sempre (deadlock).
func trabalhador(id int, wg *sync.WaitGroup, saida []string) {
    defer wg.Done()
    saida[id] = fmt.Sprintf("trabalhador %d feito", id)
}

func main() {
    fmt.Println(QuadradosParalelo([]int{1, 2, 3, 4, 5}))

    for _, r := range DividirTodos([]int{5, 0, 20, 3}) {
        fmt.Println(r.Valor, r.Erro)
    }

    var wg sync.WaitGroup
    saida := make([]string, 3)
    for i := range 3 { // range sobre int (Go >= 1.22)
        wg.Add(1)
        go trabalhador(i, &wg, saida)
    }
    wg.Wait()
    fmt.Println(saida)
}
```

Saída real (`go run -race ./e03`):

```
[1 4 9 16 25]
20 <nil>
0 divisor 1: divisão por zero
5 <nil>
33 <nil>
[trabalhador 0 feito trabalhador 1 feito trabalhador 2 feito]
```

Para pensar: (1) Muda `trabalhador(id, wg *sync.WaitGroup...)` para receber `wg sync.WaitGroup` (sem `*`) e corre: o que acontece e porquê? (2) Move o `wg.Add(1)` para dentro da goroutine: o que pode correr mal? (3) Se duas goroutines escreverem na MESMA posição, o que diz o `-race`?

A.6 Exemplo 4 - pipeline (e04)

Cada etapa (`Gerar`, `Mapear`, `Filtrar`, `Somar`) lê de um channel e escreve noutro, e fecha a sua saída quando a entrada fecha. O fecho **propaga-se** pela cadeia. `Mapear` e `Filtrar` são genéricas porque recebem a função como argumento.

```
package main

import "fmt"

// Pipeline: cada etapa lê de um channel e devolve outro.
// Quando a entrada fecha, a etapa fecha a sua saída (propaga o fim).

func Gerar(n int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for i := 1; i <= n; i++ {
            out <- i
        }
    }()
    return out
}

// Mapear aplica f a cada valor (etapa genérica).
func Mapear(in <-chan int, f func(int) int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for v := range in {
            out <- f(v)
        }
    }()
    return out
}

// Filtrar deixa passar só os valores onde ok(v) é verdadeiro.
func Filtrar(in <-chan int, ok func(int) bool) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for v := range in {
            if ok(v) {
                out <- v
            }
        }
    }()
    return out
}

// Somar é a etapa final: consome tudo e devolve um valor.
func Somar(in <-chan int) int {
    total := 0
    for v := range in {
        total += v
    }
    return total
}

func main() {
    // 1..10 -> dobro -> só os múltiplos de 3 -> soma
    dobros := Mapear(Gerar(10), func(x int) int { return x * 2 })
    mult3 := Filtrar(dobros, func(x int) bool { return x%3 == 0 })
    fmt.Println("soma:", Somar(mult3)) // 6+12+18 = 36

    // Encadear tudo numa linha (lê-se de dentro para fora):
    fmt.Println(Somar(Filtrar(Mapear(Gerar(5), func(x int) int { return x * x }),
        func(x int) bool { return x%2 == 1 }))) // 1+9+25 = 35
}
```

Saída real (`go run -race ./e04`):

```
soma: 36
35
```

Para pensar: o que acontece se uma etapa do meio esquecer o `close(out)`? Quem fica preso e porquê?

A.7 Exemplo 5 - worker pool com resultados ordenados (e05)

Versão mais completa do exercício 5: `n` trabalhadores leem de `jobs`, cada trabalho leva o seu `Indice`, e no fim os resultados são recolocados na ordem original. Mostra também como propagar erros por tarefa. Passos críticos, pela ordem: `close(jobs)` -> `wg.Wait()` -> `close(saida)` -> ler `saida`.

```
package main

import (
    "errors"
    "fmt"
    "sync"
)

type Trabalho struct {
    Indice int
    Valor  int
}

type Resultado struct {
    Indice int
    Valor  int
    Erro   error
}

// Processar corre f sobre cada tarefa com n trabalhadores e devolve os
// resultados NA ORDEM ORIGINAL (usa o Indice para recolocar).
func Processar(tarefas []int, n int, f func(int) (int, error)) []Resultado {
    jobs := make(chan Trabalho, len(tarefas))
    saida := make(chan Resultado, len(tarefas))

    var wg sync.WaitGroup
    for range n {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for t := range jobs { // acaba quando jobs fecha e esvazia
                v, err := f(t.Valor)
                saida <- Resultado{Indice: t.Indice, Valor: v, Erro: err}
            }
        }()
    }

    for i, v := range tarefas {
        jobs <- Trabalho{Indice: i, Valor: v}
    }
    close(jobs) // OBRIGATÓRIO: sem isto os range dos trabalhadores nunca acabam

    wg.Wait()
    close(saida) // só depois de TODOS os trabalhadores terminarem

    res := make([]Resultado, len(tarefas))
    for r := range saida {
        res[r.Indice] = r
    }
    return res
}

func main() {
    f := func(x int) (int, error) {
        if x < 0 {
            return 0, errors.New("negativo")
        }
        return x * 10, nil
    }
    for _, r := range Processar([]int{1, 2, -3, 4, 5, 6}, 3, f) {
        fmt.Printf("#%d valor=%d erro=%v\n", r.Indice, r.Valor, r.Erro)
    }
}
```

Saída real (`go run -race ./e05`):

```
#0 valor=10 erro=<nil>
#1 valor=20 erro=<nil>
```

```
#2 valor=0 erro=negativo
#3 valor=40 erro=<nil>
#4 valor=50 erro=<nil>
#5 valor=60 erro=<nil>
```

Para pensar: (1) Porque é que fechar `jobs` é obrigatório? (2) Porque é que `close(saida)` só pode vir DEPOIS do `wg.Wait()`? (3) Troca a ordem dos dois e corre.

A.8 Exemplo 6 - fan-in com Merge (e06)

Junta vários channels num só. Uma goroutine por entrada copia valores; uma goroutine extra espera por todas (`wg.Wait()`) e só então fecha a saída. A ordem de chegada varia, por isso o `main` ordena antes de imprimir.

```
package main

import (
    "fmt"
    "sort"
    "sync"
)

// Merge (fan-in) junta vários channels num só.
// Uma goroutine por entrada copia para a saída; uma goroutine extra
// fecha a saída quando TODAS as cópias acabarem.
func Merge(entradas ...<-chan int) <-chan int {
    out := make(chan int)
    var wg sync.WaitGroup

    copiar := func(c <-chan int) {
        defer wg.Done()
        for v := range c {
            out <- v
        }
    }

    wg.Add(len(entradas))
    for _, c := range entradas {
        go copiar(c)
    }

    go func() {
        wg.Wait()
        close(out)
    }()
    return out
}

func gerarDe(inicio, fim int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for i := inicio; i <= fim; i++ {
            out <- i
        }
    }()
    return out
}

func main() {
    var todos []int
    for v := range Merge(gerarDe(1, 3), gerarDe(10, 12), gerarDe(100, 101)) {
        todos = append(todos, v)
    }
    sort.Ints(todos) // a ordem de chegada varia; ordenamos para comparar
    fmt.Println(todos)
}
```

Saída real (`go run -race ./e06`):

```
[1 2 3 10 11 12 100 101]
```

A.9 Exemplo 7 - goroutine leak (e07)

Mostra o erro silencioso mais comum: um `select` com timeout em que a goroutine produtora fica presa a enviar para um channel **sem buffer** que ninguém lê mais. A correção é um buffer de 1. O programa conta as goroutines presas com `runtime.NumGoroutine`.

```
package main

import (
```

```
    "fmt"
    "runtime"
    "time"
)

// Armadilha: goroutine presa (leak). O channel SEM buffer obriga a goroutine
// a esperar por um receptor que nunca vem, porque o chamador desistiu.
func comLeak() {
    ch := make(chan string) // sem buffer
    go func() {
        time.Sleep(20 * time.Millisecond)
        ch <- "resultado" // fica presa aqui para sempre
    }()
    select {
    case v := <-ch:
        fmt.Println(v)
    case <-time.After(5 * time.Millisecond):
        fmt.Println("desisti")
    }
}

// Correção: buffer de 1. O envio cabe no buffer e a goroutine termina.
func semLeak() {
    ch := make(chan string, 1)
    go func() {
        time.Sleep(20 * time.Millisecond)
        ch <- "resultado"
    }()
    select {
    case v := <-ch:
        fmt.Println(v)
    case <-time.After(5 * time.Millisecond):
        fmt.Println("desisti")
    }
}

func presas(f func()) int {
    antes := runtime.NumGoroutine()
    f()
    time.Sleep(100 * time.Millisecond)
    return runtime.NumGoroutine() - antes
}

func main() {
    fmt.Println("goroutines presas (sem buffer):", presas(comLeak))
    fmt.Println("goroutines presas (com buffer):", presas(semLeak))
}
```

Saída real (`go run -race ./e07`):

```
desisti
goroutines presas (sem buffer): 1
desisti
goroutines presas (com buffer): 0
```

Regra prática: quando uma goroutine envia um resultado e o chamador pode desistir (timeout, erro), usa um channel com buffer suficiente para o envio nunca bloquear.

Parte B - Concorrência II (exercicios8)

B.1 Modelo mental

- **Data race** = duas goroutines acedem à mesma variável ao mesmo tempo e pelo menos uma escreve. `x++` não é atómico: lê, soma, grava. Duas goroutines podem ler o mesmo valor e uma das somas perde-se.
- **sync.Mutex** = uma chave. `Lock()` espera pela chave; `Unlock()` devolve-a. Só uma goroutine de cada vez na zona protegida.
- **Padrão obrigatório:** `mu.Lock()` e **logo na linha a seguir** `defer mu.Unlock()`. Já te enganaste uma vez com `defer mu.Lock()` (deadlock): relê sempre essa linha.
- Protege-se o **dado**, em TODOS os sítios que lhe tocam: leitura incluída.
- Nunca copiar um Mutex: receptores por ponteiro (`func (c *Contador)`), structs passadas por ponteiro. `go vet` avisa.
- **sync.RWMutex:** `RLock` para ler (vários ao mesmo tempo), `Lock` para escrever (exclusivo). Vale a pena quando há muito mais leituras do que escritas.
- **Dois mutexes** ao mesmo tempo -> risco de deadlock. Solução: bloquear sempre pela mesma ordem (ver `Transferir` em

e08).

- **Detetor:** `go test -race` / `go run -race`. Só vê corridas que acontecem durante a execução; mas quando vê mostra as duas linhas em conflito. Usa sempre.

B.2 select

- `select` é um `switch` para channels. Bloqueia a goroutine atual até que UM case possa avançar, executa só esse, e termina.
- **Não cria goroutines** (por isso não tem `go`). As goroutines produtoras já existem noutro lado.
- Se vários cases estiverem prontos, escolhe UM ao acaso. Não há prioridade pela ordem.
- Um case de receção está pronto se há valor, ou se o channel está fechado. Um case de envio está pronto se há receptor ou espaço no buffer.
- `case <-time.After(d)` = timeout. `default` = não bloquear (tenta e segue em frente).
- Um channel `nil` nunca fica pronto: pôr uma variável a `nil` "desliga" esse case.
- Para repetir, põe o `select` dentro de um `for`. Um `break` dentro do select só sai do select; usa `return` ou uma label para sair do `for`.

B.3 Channel ou Mutex?

- **Passar dados / coordenar fluxo** (quem faz o quê, e quando) -> channel.
- **Proteger estado simples partilhado** (contador, cache, map) -> Mutex.
- Ambos são válidos. Em `BuscarTodos` o exemplo e12 mostra as duas versões lado a lado.

B.4 Exemplo 8 - Mutex, contador e duas contas (e08)

O contador faz 1000 goroutines x 100 incrementos = 100000 certo. As duas contas mostram como **evitar deadlock com dois mutexes**: transferências em sentidos opostos (A->B e B->A) a bloquear em ordem diferente ficariam presas; ordenando pelo ID, ficam sempre pela mesma ordem.

```
package main

import (
    "errors"
    "fmt"
    "sync"
)

// ---- Contador: a regra de ouro é Lock + defer Unlock na linha seguinte.
type Contador struct {
    mu    sync.Mutex
    valor int
}

func (c *Contador) Incrementar() {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.valor++ // ler-somar-gravar: sem lock, duas goroutines perdem somas
}

func (c *Contador) Valor() int {
    c.mu.Lock() // também a LEITURA precisa de lock
    defer c.mu.Unlock()
    return c.valor
}

// ---- Duas contas: bloquear DOIS mutexes pode dar deadlock.
// A faz Lock(A) e espera B; B faz Lock(B) e espera A -> ficam presas.
// Solução: bloquear SEMPRE pela mesma ordem (aqui, pelo ID).
var ErrSaldo = errors.New("saldo insuficiente")

type Conta struct {
    mu    sync.Mutex
    ID    int
    Saldo int
}

func Transferir(de, para *Conta, valor int) error {
    if de == para {
        return errors.New("mesma conta")
    }
    primeiro, segundo := de, para
    if para.ID < de.ID {
```

```
        primeiro, segundo = para, de
    }
    primeiro.mu.Lock()
    defer primeiro.mu.Unlock()
    segundo.mu.Lock()
    defer segundo.mu.Unlock()

    if de.Saldo < valor {
        return ErrSaldo
    }
    de.Saldo -= valor
    para.Saldo += valor
    return nil
}

func main() {
    var c Contador
    var wg sync.WaitGroup
    for range 1000 {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for range 100 {
                c.Incrementar()
            }
        }()
    }
    wg.Wait()
    fmt.Println("contador:", c.Valor()) // 100000

    a := &Conta{ID: 1, Saldo: 1000}
    b := &Conta{ID: 2, Saldo: 1000}
    for range 100 {
        wg.Add(2)
        go func() { defer wg.Done(); Transferir(a, b, 10) }() // A -> B
        go func() { defer wg.Done(); Transferir(b, a, 10) }() // B -> A (sentido oposto)
    }
    wg.Wait()
    fmt.Println("total:", a.Saldo+b.Saldo) // 2000: nada se perde

    fmt.Println(Transferir(a, b, 999999)) // saldo insuficiente
}
```

Saída real (`go run -race ./e08`):

```
contador: 100000
total: 2000
saldo insuficiente
```

Para pensar: (1) Tira o `Lock` de `Valor()` e corre com `-race`: o que diz? (2) Em `Transferir`, remove a ordenação por ID e aumenta o número de goroutines: consegues provocar deadlock? (3) Porque é que `de == para` é tratado à parte? (pista: bloquear o mesmo mutex duas vezes.)

B.5 Exemplo 9 - Cache genérica com RWMutex (e09)

Cache com genéricos (`Cache[V any]`), `RLock` para ler e `Lock` para escrever. `ObterOuCalcular` usa **verificação dupla**: tenta ler com `RLock`; se falhar, sobe para `Lock` e volta a verificar, porque outra goroutine pode ter chegado primeiro. Resultado: 50 goroutines pedem a mesma chave e o cálculo corre uma só vez.

```
package main

import (
    "fmt"
    "sync"
)

// Cache genérica com RWMutex: muitos leitores OU um escritor.
type Cache[V any] struct {
    mu    sync.RWMutex
    dados map[string]V
}

func NovaCache[V any]() *Cache[V] {
    return &Cache[V]{dados: make(map[string]V)} // map nil não aceita escrita
}

func (c *Cache[V]) Obter(k string) (V, bool) {
    c.mu.RLock() // leitura partilhada
    defer c.mu.RUnlock()
    v, ok := c.dados[k]
    return v, ok
```

```
}

func (c *Cache[V]) Por(k string, v V) {
    c.mu.Lock() // escrita exclusiva
    defer c.mu.Unlock()
    c.dados[k] = v
}

func (c *Cache[V]) Apagar(k string) {
    c.mu.Lock()
    defer c.mu.Unlock()
    delete(c.dados, k)
}

func (c *Cache[V]) Len() int {
    c.mu.RLock()
    defer c.mu.RUnlock()
    return len(c.dados)
}

// ObterOuCalcular usa "verificação dupla": o caminho rápido só lê (RLock);
// se falhar, sobe para Lock e volta a verificar, porque entre o RUnlock e o
// Lock outra goroutine pode ter chegado primeiro.
func (c *Cache[V]) ObterOuCalcular(k string, calcular func() V) V {
    if v, ok := c.Obter(k); ok {
        return v
    }
    c.mu.Lock()
    defer c.mu.Unlock()
    if v, ok := c.dados[k]; ok { // segunda verificação
        return v
    }
    v := calcular()
    c.dados[k] = v
    return v
}

func main() {
    c := NovaCache[int]()
    c.Por("a", 1)
    v, ok := c.Obter("a")
    fmt.Println(v, ok)
    _, ok = c.Obter("zzz")
    fmt.Println(ok)

    calculos := 0 // só mexido dentro de calcular(), que corre sob Lock
    var wg sync.WaitGroup
    for range 50 {
        wg.Add(1)
        go func() {
            defer wg.Done()
            c.ObterOuCalcular("pesado", func() int {
                calculos++
                return 42
            })
        }()
    }
    wg.Wait()
    fmt.Println("calculos:", calculos, "len:", c.Len()) // 1 vez só

    c.Apagar("a")
    fmt.Println("len:", c.Len())
}
```

Saída real (`go run -race ./e09`):

```
1 true
false
calculos: 1 len: 2
len: 1
```

Para pensar: (1) O que acontece se tirares a segunda verificação (`if v, ok := c.dados[k]`)? Quantos cálculos fazes? (2) Porque é que `NovaCache` tem de usar `make`? (3) Não dá para "subir" de `RLock` para `Lock` sem largar o primeiro. Porquê?

B.6 Exemplo 10 - select em todas as formas (e10)

Timeout, não-bloqueante (`default`), desligar cases com `nil` (`Juntar`) e `for` + `select` + canal `parar` com `time.Ticker`. Este último é o padrão para parar uma goroutine de fora: `close(parar)` acorda o case `<-parar`.

```
package main
```

```
import (
    "errors"
    "fmt"
    "sort"
    "time"
)

var ErrTimeout = errors.New("timeout")

// 1) select com timeout.
func EsperarOuTimeout(ch <-chan int, d time.Duration) (int, error) {
    select {
    case v := <-ch:
        return v, nil
    case <-time.After(d):
        return 0, ErrTimeout
    }
}

// 2) default = não bloqueia (tenta e segue em frente).
func TentarReceber(ch <-chan int) (int, bool) {
    select {
    case v := <-ch:
        return v, true
    default:
        return 0, false
    }
}

func TentarEnviar(ch chan<- int, v int) bool {
    select {
    case ch <- v:
        return true
    default: // buffer cheio ou ninguém a ler
        return false
    }
}

// 3) channel nil NUNCA fica pronto: serve para "desligar" um case.
// Lê de a e b até os dois fecharem.
func Juntar(a, b <-chan int) []int {
    var res []int
    for a != nil || b != nil {
        select {
        case v, ok := <-a:
            if !ok {
                a = nil // desliga este case
                continue
            }
            res = append(res, v)
        case v, ok := <-b:
            if !ok {
                b = nil
                continue
            }
            res = append(res, v)
        }
    }
    return res
}

func emitir(vals ...int) <-chan int {
    ch := make(chan int)
    go func() {
        defer close(ch)
        for _, v := range vals {
            ch <- v
        }
    }()
    return ch
}

// 4) for + select + canal "done" para parar uma goroutine de fora.
func contarTicks(parar <-chan struct{}) int {
    t := time.NewTicker(10 * time.Millisecond)
    defer t.Stop()
    n := 0
    for {
        select {
        case <-t.C:
            n++
        case <-parar:
            return n
```

```
        }
    }
}

func main() {
    rapido := make(chan int, 1)
    rapido <- 7
    fmt.Println(EsperarOuTimeout(rapido, time.Second))
    fmt.Println(EsperarOuTimeout(make(chan int), 20*time.Millisecond))

    vazio := make(chan int, 1)
    fmt.Println(TentarReceber(vazio))
    fmt.Println(TentarEnviar(vazio, 1), TentarEnviar(vazio, 2)) // true false

    r := Juntar(emitir(1, 2, 3), emitir(10, 20))
    sort.Ints(r)
    fmt.Println(r)

    parar := make(chan struct{})
    res := make(chan int)
    go func() { res <- contarTicks(parar) }()
    time.Sleep(65 * time.Millisecond)
    close(parar)
    fmt.Println("ticks >= 4:", <-res >= 4)
}
```

Saída real (`go run -race ./e10`):

```
7 <nil>
0 timeout
0 false
true false
[1 2 3 10 20]
ticks >= 4: true
```

Para pensar: (1) Em `Juntar`, o que acontece sem o `a = nil`? (dica: um channel fechado está sempre "pronto".) (2) Porque é que o `defer t.Stop()` é importante num Ticker?

B.7 Exemplo 11 - primeiro a responder e corrida com N fontes (e11)

`PrimeiroAResponder` é o do exercício 4. `Corrida` generaliza: N fontes lançadas em paralelo, um channel com buffer `len(fontes)` para as lentas nunca ficarem presas, e um limite de tempo. Devolve o mais rápido ou `ErrTimeout`.

```
package main

import (
    "errors"
    "fmt"
    "time"
)

var ErrTimeout = errors.New("timeout")

// PrimeiroAResponder (ex. 4): devolve o primeiro valor que chegar.
func PrimeiroAResponder(a, b <-chan string) string {
    select {
    case v := <-a:
        return v
    case v := <-b:
        return v
    }
}

// Corrida generaliza para N fontes + limite de tempo.
// O channel tem buffer len(fontes): as goroutines mais lentas conseguem
// enviar e terminar mesmo que ninguém leia (sem goroutine leak).
func Corrida(fontes []func() string, limite time.Duration) (string, error) {
    ch := make(chan string, len(fontes))
    for _, f := range fontes {
        go func() { ch <- f() }()
    }
    select {
    case v := <-ch:
        return v, nil
    case <-time.After(limite):
        return "", ErrTimeout
    }
}

func fonte(nome string, d time.Duration) func() string {
    return func() string {
        time.Sleep(d)
```

```
        return nome
    }
}

func main() {
    a := make(chan string, 1)
    b := make(chan string, 1)
    go func() { time.Sleep(50 * time.Millisecond); a <- "lento" }()
    go func() { time.Sleep(5 * time.Millisecond); b <- "rápido" }()
    fmt.Println(PrimeiroAResponder(a, b))

    fontes := []func() string{
        fonte("A", 30*time.Millisecond),
        fonte("B", 10*time.Millisecond),
        fonte("C", 20*time.Millisecond),
    }
    fmt.Println(Corrida(fontes, time.Second))
    fmt.Println(Corrida(fontes, 2*time.Millisecond))
}
```

Saída real (`go run -race ./e11`):

```
rápido
B <nil>
 timeout
```

Para pensar: se o buffer fosse 0, quantas goroutines ficariam presas depois de a função devolver?

B.8 Exemplo 12 - BuscarTodos de 3 maneiras (e12)

1) **Mutex**: as goroutines escrevem no map; o `buscar` fica FORA do lock (só o acesso ao map é protegido, senão perdes o paralelismo). 2) **Channel**: as goroutines só enviam; uma só goroutine escreve no map, logo não há partilha e não há Mutex. 3) **Limite de concorrência** com um semáforo (`chan struct{}` com buffer): `sem <- struct{}{}` pede lugar, `<-sem` devolve. O `main` prova que nunca houve mais de 3 ao mesmo tempo, usando `sync/atomic`.

```
package main

import (
    "errors"
    "fmt"
    "sort"
    "sync"
    "sync/atomic"
    "time"
)

type Buscar func(url string) (string, error)

// Versão 1 — Mutex: cada goroutine escreve no map/slice partilhados.
// O download (buscar) fica FORA do lock: só o acesso aos dados é protegido.
func BuscarMutex(urls []string, buscar Buscar) (map[string]string, []string) {
    res := make(map[string]string)
    var falhas []string
    var mu sync.Mutex
    var wg sync.WaitGroup

    for _, url := range urls {
        wg.Add(1)
        go func() {
            defer wg.Done()
            conteudo, err := buscar(url)
            mu.Lock()
            defer mu.Unlock()
            if err != nil {
                falhas = append(falhas, url)
                return
            }
            res[url] = conteudo
        }()
    }
    wg.Wait()
    return res, falhas
}

// Versão 2 — channel: as goroutines só ENVIAM; uma única goroutine (a
// função) escreve no map. Sem Mutex porque nada é partilhado.
func BuscarCanal(urls []string, buscar Buscar) (map[string]string, []string) {
    type resp struct {
        url, conteudo string
        err           error
    }
```

```
    ch := make(chan resp, len(urls))
    var wg sync.WaitGroup

    for _, url := range urls {
        wg.Add(1)
        go func() {
            defer wg.Done()
            c, err := buscar(url)
            ch <- resp{url, c, err}
        }()
    }
    go func() { wg.Wait(); close(ch) }()

    res := make(map[string]string)
    var falhas []string
    for r := range ch {
        if r.err != nil {
            falhas = append(falhas, r.url)
        } else {
            res[r.url] = r.conteudo
        }
    }
    return res, falhas
}

// Versão 3 — limite de concorrência com um "semáforo" (channel com buffer).
// No máximo `limite` downloads ao mesmo tempo.
func BuscarLimitado(urls []string, limite int, buscar Buscar) (map[string]string, []string) {
    sem := make(chan struct{}, limite)
    res := make(map[string]string)
    var falhas []string
    var mu sync.Mutex
    var wg sync.WaitGroup

    for _, url := range urls {
        wg.Add(1)
        go func() {
            defer wg.Done()
            sem <- struct{}{}        // pede um lugar (bloqueia se estiver cheio)
            defer func() { <-sem }() // devolve o lugar
            c, err := buscar(url)
            mu.Lock()
            defer mu.Unlock()
            if err != nil {
                falhas = append(falhas, url)
            } else {
                res[url] = c
            }
        }()
    }
    wg.Wait()
    return res, falhas
}

func main() {
    buscar := func(u string) (string, error) {
        time.Sleep(10 * time.Millisecond)
        if u == "mau1" || u == "mau2" {
            return "", errors.New("falhou")
        }
        return "conteúdo de " + u, nil
    }
    urls := []string{"a", "mau1", "b", "c", "mau2"}

    for nome, f := range map[string]func([]string, Buscar) (map[string]string, []string){
        "mutex": BuscarMutex, "canal": BuscarCanal,
    } {
        res, falhas := f(urls, buscar)
        sort.Strings(falhas)
        fmt.Println(nome, len(res), falhas)
    }

    // Prova do limite: medimos quantos buscar() correm ao mesmo tempo.
    var ativos, maximo atomic.Int32
    medido := func(u string) (string, error) {
        n := ativos.Add(1)
        for {
            m := maximo.Load()
            if n <= m || maximo.CompareAndSwap(m, n) {
                break
            }
        }
        time.Sleep(10 * time.Millisecond)
        ativos.Add(-1)
```

```
        return u, nil
    }
    muitos := []string{"1", "2", "3", "4", "5", "6", "7", "8", "9", "10"}
    BuscarLimitado(muitos, 3, medido)
    fmt.Println("máximo simultâneo <= 3:", maximo.Load() <= 3)
}
```

Saída real (`go run -race ./e12`):

```
mutex 3 [mau1 mau2]
canal 3 [mau1 mau2]
máximo simultâneo <= 3: true
```

Para pensar: (1) Na versão do channel, porque é que há uma goroutine só para `wg.Wait(); close(ch)`? (2) Qual das duas versões preferes e porquê? (a pergunta 4 do CONTEXTO.md). (3) O que acontece se o semáforo tiver tamanho 0?

B.9 Exemplo 13 - integrador (e13)

Junta tudo: serviço com cache (`RWMutex`), limite de concorrência (semáforo), timeout por pedido (`select` + `time.After`, com buffer 1 para não haver leak), estatísticas com `Mutex`, e `WaitGroup` para esperar por todos. Lê-o devagar, uma função de cada vez. Na ronda 2 os que tiveram sucesso vêm da cache (cache=3).

```
package main

import (
    "errors"
    "fmt"
    "sort"
    "sync"
    "time"
)

// Integrador: junta Mutex, RWMutex, select, WaitGroup e semáforo.
// Um "serviço" que busca URLs em paralelo, com cache, limite de
// concorrência, timeout por pedido e estatísticas.

var ErrTimeout = errors.New("timeout")

type Estatisticas struct {
    mu                                sync.Mutex
    Sucessos, Falhas, Timeouts, DaCache int
}

func (e *Estatisticas) Somar(campo *int) {
    e.mu.Lock()
    defer e.mu.Unlock()
    *campo++
}

type Servico struct {
    buscar  func(string) (string, error)
    timeout time.Duration
    sem     chan struct{}
    cacheMu sync.RWMutex
    cache   map[string]string
    Estat   Estatisticas
}

func NovoServico(buscar func(string) (string, error), limite int, timeout time.Duration) *Servico {
    return &Servico{
        buscar:  buscar,
        timeout: timeout,
        sem:     make(chan struct{}, limite),
        cache:   make(map[string]string),
    }
}

// comTimeout: o buffer 1 evita que a goroutine fique presa se o select
// já tiver desistido.
func (s *Servico) comTimeout(url string) (string, error) {
    type resp struct {
        c   string
        err error
    }
    ch := make(chan resp, 1)
    go func() {
        c, err := s.buscar(url)
        ch <- resp{c, err}
    }()
    select {
    case r := <-ch:
```

```
        return r.c, r.err
    case <-time.After(s.timeout):
        return "", ErrTimeout
    }
}

func (s *Servico) um(url string) (string, error) {
    s.cacheMu.RLock()
    c, ok := s.cache[url]
    s.cacheMu.RUnlock()
    if ok {
        s.Estat.Somar(&s.Estat.DaCache)
        return c, nil
    }

    s.sem <- struct{}{}
    defer func() { <-s.sem }()

    c, err := s.comTimeout(url)
    switch {
    case errors.Is(err, ErrTimeout):
        s.Estat.Somar(&s.Estat.Timeouts)
    case err != nil:
        s.Estat.Somar(&s.Estat.Falhas)
    default:
        s.Estat.Somar(&s.Estat.Sucessos)
        s.cacheMu.Lock()
        s.cache[url] = c
        s.cacheMu.Unlock()
    }
    return c, err
}

// Todos devolve conteúdo por URL e o erro de cada URL que falhou.
func (s *Servico) Todos(urls []string) (map[string]string, map[string]error) {
    ok := make(map[string]string)
    erros := make(map[string]error)
    var mu sync.Mutex
    var wg sync.WaitGroup

    for _, url := range urls {
        wg.Add(1)
        go func() {
            defer wg.Done()
            c, err := s.um(url)
            mu.Lock()
            defer mu.Unlock()
            if err != nil {
                erros[url] = err
            } else {
                ok[url] = c
            }
        }()
    }
    wg.Wait()
    return ok, erros
}

func main() {
    buscar := func(u string) (string, error) {
        switch u {
        case "lento":
            time.Sleep(300 * time.Millisecond)
        case "mau":
            return "", errors.New("erro de rede")
        default:
            time.Sleep(10 * time.Millisecond)
        }
        return "conteúdo de " + u, nil
    }
    s := NovoServico(buscar, 2, 60*time.Millisecond)
    urls := []string{"a", "b", "c", "mau", "lento"}

    for ronda := 1; ronda <= 2; ronda++ {
        ok, erros := s.Todos(urls)
        var chaves []string
        for u := range erros {
            chaves = append(chaves, u)
        }
        sort.Strings(chaves)
        fmt.Printf("ronda %d: ok=%d erros=%v\n", ronda, len(ok), chaves)
    }
    e := &s.Estat
    fmt.Printf("sucessos=%d falhas=%d timeouts=%d cache=%d\n",
```

```
        e.Sucessos, e.Falhas, e.Timeouts, e.DaCache)
}
```

Saída real (`go run -race ./e13`):

```
ronda 1: ok=3 erros=[lento mau]
ronda 2: ok=3 erros=[lento mau]
sucessos=3 falhas=2 timeouts=2 cache=3
```

Para pensar: (1) Em `um`, porque é que o `RUnlock` é chamado antes do `s.sem <-`? (2) Duas goroutines podem pedir o MESMO url ao mesmo tempo e ambas falhar a cache. Como usarias o padrão do e09 para evitar? (3) Porque é que `Estatisticas.Somar` recebe um ponteiro para o campo?

Parte C - Dúvidas que apareceram

C.1 select vs switch, e a palavra go

- É **select**, não switch. `switch` compara valores; `select` espera por channels.
- O `select` **não precisa de** `go` porque não cria goroutines. Corre na goroutine atual e bloqueia-a até haver um case pronto.
- O `select` executa **só um case** de cada vez, mesmo que vários estejam prontos (escolhe um ao acaso). O que corre "ao mesmo tempo" são os produtores, em goroutines diferentes.
- Para receber os DOIS valores, tens de ter dois recebimentos (por exemplo um `for` com 2 voltas de `select`).

C.2 for _, url := range urls

- `urls` é uma **lista** de vários textos. _ é o índice (0, 1, 2...), que ignoras. `url` é **um** elemento em cada volta, o endereço ("a", "mau1"...), e NÃO o conteúdo descarregado.
- O conteúdo é o que `buscar(url)` devolve. No map: **chave = url, valor = conteúdo** (`res[url] = conteudo`).
- As falhas são uma lista de **URLs** (`[]string`), não de erros: `falhas = append(falhas, url)`.

C.3 Erros comuns que já tiveste (lista de verificação)

- `defer c.mu.Lock()` em vez de `Unlock()` -> deadlock. Relê a linha do `defer`.
- `sync.WaitGroup` e `sync.Mutex` são TIPOS: declara com `var wg sync.WaitGroup`, não com `wg := sync.WaitGroup`.
- Os métodos têm maiúscula e nome certo: `wg.Done()` (não `close`), `mu.Lock()`/`Unlock()` (não `lock`).
- `if err != nil` invertido: o sucesso é `err == nil`.
- `wg.Wait()` dentro do ciclo anula o paralelismo: vai DEPOIS do ciclo.
- `Lock` sem `Unlock`: as outras goroutines ficam presas para sempre.
- Map `nil` (`var m map[string]string`) não aceita escrita: usa `make`.
- Guardar o texto do erro quando se pedia o URL que falhou.
- `defer` sem o `defer` (`f.Close()` solto) fecha logo.
- Misturar duas abordagens (channels E mutex) para o mesmo problema.

Parte D - Desafios (sem solução, para treinares)

Faz estes sem copiar. Cada um usa só o que viste nos exemplos. Valida com `go run -race`.

- **D1.** Escreve `ContarPalavras(textos []string) map[string]int` com uma goroutine por texto e um Mutex para o map partilhado. Depois reescreve com channels, sem Mutex.
- **D2.** Pipeline: `Gerar(n)` -> `Cubos` -> `SoMultiplosDe(k)` -> `Somar`. Testa com n=0 e n=1.
- **D3.** Worker pool que processa uma lista de URLs falsos com n=3 trabalhadores e devolve os resultados na ordem original, parando de aceitar trabalho se um erro ocorrer (pista: um canal `parar`).
- **D4.** `Contador` com `RWMutex` e um método `Ler()` com `RLock`. Prova com `-race` que 100 leitores e 10 escritores não dão corrida.
- **D5.** `EsperarTodosOuTimeout(chs []<-chan int, d time.Duration) ([]int, error)`: recolhe um valor de cada channel ou devolve `ErrTimeout`.
- **D6.** Semáforo: `ExecutarLimitado(tarefas []func(), limite int)` que corre todas, no máximo `limite` ao mesmo tempo.

- **D7.** Provoca de propósito (e depois corrige): (a) uma data race num contador; (b) um deadlock com dois mutexes; (c) um goroutine leak. Descreve a mensagem que Go dá em cada.
- **D8.** Cache com expiração: `Por(k, v, ttl)`; `Obter` ignora entradas expiradas. Protege com `RWMutex`.

Parte E - Autoavaliação

Responde por palavras tuas, sem consultar. Se falhares alguma, volta ao exemplo indicado.

- Qual a diferença entre channel com e sem buffer? (e01)
- O que devolve receber de um channel fechado? E enviar para um? (e02)
- Porque é que `wg.Add(1)` vai antes do `go`? (e03)
- Quem deve fechar um channel? (e02, e05)
- Porque é que a ordem `close(jobs)`, `wg.Wait()`, `close(saida)` é importante? (e05)
- O que é um goroutine leak e como se evita com um buffer? (e07, e11)
- Porque é que `Valor()` também precisa de lock? (e08)
- Como se evita deadlock com dois mutexes? (e08)
- Quando prefiro RWMutex a Mutex? (e09)
- O que acontece num select se dois cases estiverem prontos? (B.2)
- Para que serve um channel `nil` num select? (e10)
- Quando é que prefiro channel de resultados em vez de Mutex? (e12)

Parte F - Cheat sheet (uma página)

```
go f(x)                    // lança goroutine
ch := make(chan T)        // sem buffer
ch := make(chan T, n)     // com buffer n
ch <- v                   // enviar
v := <-ch                 // receber
v, ok := <-ch             // ok == false: fechado e vazio
close(ch)                 // só quem envia, uma vez
for v := range ch { }     // até fechar
func f(in <-chan T) <-chan T   // só lê / só envia

var wg sync.WaitGroup
wg.Add(1); go func(){ defer wg.Done(); ... }(); wg.Wait()

var mu sync.Mutex
mu.Lock(); defer mu.Unlock()        // sempre juntos
var rw sync.RWMutex
rw.RLock(); defer rw.RUnlock()      // leitura partilhada
rw.Lock();  defer rw.Unlock()       // escrita exclusiva

select {
case v := <-a:              ...
case b <- x:                ...
case <-time.After(d):       ...   // timeout
default:                    ...   // não bloquear
}

go vet ./...      gofmt -l .      go test -race ./...
```

Parte G - Plano de revisão

- **Quinta:** A.1 e A.2 (30 min). Exemplos e01-e04: copiar, correr, mudar uma coisa (1h). Perguntas de A.3 a A.6.
- **Quinta (depois):** e05-e07 e desafios D1 e D2. Anota o que não percebeste.
- **Sexta:** B.1 a B.3 (30 min). Exemplos e08-e11 (1h). Depois e12 e e13 com calma.
- **Sexta (final):** desafios D3 a D7 e a autoavaliação da Parte E. Se falhares 3 ou mais, repete esse exemplo.
- **Sábado/Domingo:** projeto. Não precisas de concorrência na CLI (só se fizeres os extras da etapa 5), mas o hábito de testar com `-race` mantém-se.